PROJECT HANDOFF / TEAM PLAN

Last Mile

Task distribution and integration plan

A two-person delivery plan for the secure family vault MVP.

Team Lead	Developer	Current state
Mayank Sharma	Shyam	Scaffold pushed to GitHub

How we will work

The work is split into two parallel tracks: Mayank owns browser-side cryptography and the recovery workflow engine; Shyam owns the data foundation and product experience. Both tracks meet at a small, agreed API and type contract, then get validated together through the end-to-end demo.

MAYANK | CORE SECURITY & WORKFLOW

Vault encryption and key handling; 2-of-3 recovery; recovery state transitions; audit-chain verification; integration and security demo.

SHYAM | DATA FOUNDATION & PRODUCT FLOW

Supabase schema and access controls; app shell and responsive screens; asset/nominee workflows; persistence APIs and UX validation.

Rules of engagement

- The scaffold and first commit are already done. Start by pulling the latest repository and running the existing checks before editing.
- Use the shared docs as the requirements baseline: Project Brief, PRD, Tech Research, TRD, App Flow, Backend Schema, API Contract, Security Threat Model, Implementation Plan, Test Plan and Architecture Decisions.
- Where an older draft assigns a task differently, this allocation is the current working ownership plan. The security requirements themselves do not change.
- Use synthetic data only. Never commit real documents, passphrases, recovery cards, account identifiers or environment secrets.

01 / MAYANK SHARMA

Workstream A: Vault security and recovery engine

Own the security-sensitive logic end-to-end, expose clear interfaces for the UI, and prove the core recovery path with automated tests.

A1. Browser-side cryptography

- Implement client-only AES-256-GCM encryption and decryption for the vault payload, with a fresh 12-byte nonce for each encryption operation and context-bound additional authenticated data (AAD).
- Generate a random 256-bit data-encryption key (DEK); derive the owner key from a passphrase using the KDF specified in the TRD; wrap and unwrap the DEK in the browser.
- Use the specified Shamir library to split the DEK into 3 shares with a threshold of 2. Do not implement cryptographic primitives from scratch.
- Implement versioned recovery-card encoding/decoding and validation: malformed cards, wrong vault IDs, duplicate share indices, incompatible versions and insufficient shares must fail safely.
- Keep plaintext, passphrase, DEK, KEK and share bytes out of API requests, persistent storage, analytics and logs. Clear temporary references as best effort.

A2. Recovery state machine and server rules

- Implement server-authoritative transitions for check-in, grace period, recovery request, distinct nominee approvals, unlock window, cooling-off and owner cancellation.
- Own the recovery workflow API routes described in the API Contract, including approval/decline, cancellation, recovery session, client-reported completion and demo-only clock advancement.
- Prevent duplicate nominee approvals from counting twice. Use transaction-safe checks, idempotency where appropriate, and deterministic handling of expiry/cancellation races.
- Keep the demo clock out of production behavior. Use server/database time for real transitions, and label the simulated-time control clearly as demo-only.

A3. Audit integrity, tests and integration

- Implement canonical audit-event hashing and hash-chain verification; show a clear passed/failed integrity result and document the known limitations of an unanchored hash chain.
- Own the crypto, recovery and audit domain tests: round-trip, tampering, wrong key, one-share failure, valid two-share recovery, duplicate/invalid shares, duplicate approvals, forbidden transitions and cancellation races.
- Integrate both workstreams at the agreed contracts, review the final network/database privacy proof, and lead the technical walkthrough for the demo.

Suggested file ownership

Mayank: src/lib/crypto/**, src/lib/recovery/**, src/lib/audit/**, the recovery/check-in/workflow/demo-clock API route folders, plus their tests. Adapt paths to the scaffold rather than creating duplicate modules.

Primary references: 03_TRD.md, 04_APP_FLOW.md, 06_API_CONTRACT.md, 07_SECURITY_THREAT_MODEL.md, 9_TEST_PLAN.md, 10_ARCHITECTURE_DECISIONS.md.

02 / SHYAM

Workstream B: Data foundation and product experience

Build the secure persistence layer and the user-facing flows that make the vault easy to understand and operate.

B1. Supabase data foundation and authorization

- Create and maintain migrations for the documented tables: profiles, vaults, nominees, recovery_requests, recovery_approvals and audit_events. Add required constraints, unique rules and indexes.
- Implement RLS policies and grants based on the threat model. Prove that one signed-in user cannot read or modify another user's records.
- Implement the normal data persistence endpoints for vault creation/read, encrypted-envelope save/load, nominee management and invite acceptance. Validate every input and authorize every request on the server.
- Persist only encrypted envelope fields, wrapped-key envelope, KDF parameters, workflow metadata and permitted non-secret audit fields. Never add columns for plaintext assets or recovery-share bytes.

B2. Product UI and core user journeys

- Implement the approved Concept C direction from Last_Mile_UI_UX_Reference_Pack/reference/Last_Mile_UI_UX_Concept_C_Approved.png: editorial scrapbook/family archive, paper textures used sparingly, readable typography and clear operational controls. Treat the image as visual direction; agree any additional page, token or component details together before expanding scope.
- Build the shared app shell, responsive navigation, buttons, form elements, alerts, confirmation dialogs, loading states and empty/error states.
- Build public landing, sign-in/onboarding, dashboard, asset inventory and add/edit forms, nominee list/management, check-in screen, recovery timeline, nominee approval screen, recovery UI surface and activity log.
- Build recovery screens against Mayank's agreed interfaces. The UI can gather/import recovery cards locally but must not upload or log card/share bytes.
- Prioritize the end-to-end happy path, then responsive behavior and polish. Use only the synthetic demo records in the project docs.

B3. UI, data and authorization quality

- Add component/flow tests for forms, route loading and error states, auth guards, mobile navigation and critical user journeys.
- Add migration/RLS tests or repeatable verification steps for cross-user denial, nominee authorization and server-side input validation.
- Verify keyboard use, visible focus, field-level errors and small-screen layouts; do not hide critical states behind color or hover.

Suggested file ownership

Shyam: supabase/migrations/**, data-access helpers, normal vault/envelope/nominee/invite API route folders, shared product pages/components/styles, plus their tests. Coordinate before editing shared layout, global styles or dependency manifests.

Primary references: 01_PRD.md, 05_BACKEND_SCHEMA.md, 06_API_CONTRACT.md, 8_IMPLEMENTATION_PLAN.md, 9_TEST_PLAN.md and the approved image.

03 / CONTRACTS & FILE BOUNDARIES

Define the seams before parallel coding

Agree on request/response types and database field names in one short pairing session. Then keep each workstream inside its ownership boundary and integrate through small pull requests.

Area	Owner	Logical endpoints / boundaries
Vault persistence	Shyam	POST /api/vaults GET /api/vaults/:vaultId PUT /api/vaults/:vaultId/envelope
Nominee management	Shyam	POST /api/vaults/:vaultId/nominees POST /api/invites/accept
Check-in and recovery state	Mayank	POST /api/vaults/:vaultId/check-in POST /api/vaults/:vaultId/recovery-requests decision / cancel / session / client-completion routes
Workflow jobs and demo clock	Mayank	POST /api/internal/advance-workflows POST /api/dev/demo-clock (demo environment only)
Audit chain and verification	Mayank	Canonical audit hashing, verification result and GET /api/vaults/:vaultId/audit read surface; coordinate schema with Shyam.
DB migrations and RLS	Shyam	Schema, indexes, constraints, grants, policies and cross-user tests. Mayank reviews security-sensitive assumptions.
Client cryptography	Mayank	Crypto library and browser modules only; no API or DB path accepts passphrases, plaintext asset objects, DEKs or recovery shares.

Shared contracts to agree before coding

Contract	Agreement
API types	Use 06_API_CONTRACT.md as the endpoint baseline. Agree on a shared typed schema/validation module; no undocumented payload shapes.
Database envelope	Use 03_TRD.md and 05_BACKEND_SCHEMA.md. Store ciphertext envelope + wrapped DEK + KDF metadata; never store decrypted payload or any share.
State transitions	Only the server-side recovery service changes lifecycle state. UI buttons request transitions; they do not decide whether a transition is valid.
Errors and security	Agree consistent error codes and loading/empty/failed UI states. No secret material in logs, URLs, screenshots, analytics or API bodies.

File-collision rule

Avoid simultaneous edits to package.json/lockfile, shared types, global CSS, root layout, and database schema. If a shared file must change, announce it in chat, make the change in one branch, and let the other branch rebase on it.

04 / DELIVERY & ACCEPTANCE

Integrate in thin, testable slices

The objective is one repeatable secure demo, not disconnected pages or a long feature list.

Step	Mayank	Shyam	Joint exit condition
1. Sync contracts	Confirm crypto envelope and recovery APIs.	Confirm table fields, RLS and UI data needs.	Route shapes and shared types agreed before parallel feature coding.
2. Build in parallel	Crypto module, recovery state service and audit chain with unit tests.	Migrations/RLS, persistence APIs, app shell and primary pages.	Each branch passes lint, typecheck and relevant tests.
3. Integrate	Wire local crypto and state transitions to the API/UI.	Wire the UI to real endpoints and remove fake success states.	No duplicate source-of-truth logic; errors and expired states are visible.
4. Harden	Run crypto, transition, audit and recovery edge cases.	Run auth/RLS, form, mobile and accessibility checks.	Clean checkout build and end-to-end demo both pass.

Definition of done: minimum security gates

- Synthetic vault content encrypts in the browser; database inspection shows only ciphertext envelope, wrapped DEK and permitted metadata.
- Modified ciphertext fails authentication; one share cannot recover the DEK; two valid shares from the same split can recover; duplicate or mismatched shares are rejected.
- Nominee approvals count only once per distinct authorized nominee; invalid state transitions are rejected; owner cancellation works during the permitted cooling-off stage.
- Cross-user reads/writes are denied by authorization and RLS; recovery cards and raw shares never reach the backend.
- Audit-chain verification reports tampering when a retained prior hash/checkpoint makes it detectable; the demo does not claim protection against every privileged database rewrite.
- Lint, typecheck, tests and production build pass from a clean checkout; the demo uses no real financial or identity data.

Demo sequence

1. Create a vault with synthetic data. 2. Save encrypted payload and inspect the database/network. 3. Advance the demo clock through the configured recovery states. 4. Capture approvals from two distinct nominees. 5. Import two recovery cards into the browser and decrypt locally. 6. Reset the demo and show the owner cancelling a false trigger. 7. Show the test report and the threat-model summary.

Daily handoff

Each person posts: completed, next, blocked by, and files/routes changed. Keep PRs small. Never merge a branch that leaves the build red or bypasses authorization for the demo.

If time is short, cut OCR, email automation, advanced illustration polish and optional features first. Keep encryption tests, RLS, duplicate-approval protection, cancellation and the ciphertext-only proof.

Reference the files currently present in the repository: 00_PROJECT_BRIEF.md, 01_PRD.md, 02_TECH_RESEARCH_AND_STACK.md, 03_TRD.md, 04_APP_FLOW.md, 05_BACKEND_SCHEMA.md, 06_API_CONTRACT.md, 07_SECURITY_THREAT_MODEL.md, 8_IMPLEMENTATION_PLAN.md, 9_TEST_PLAN.md, 10_ARCHITECTURE_DECISIONS.md, and the approved Concept C image inside Last_Mile_UI_UX_Reference_Pack/reference/. Do not assume removed documents exist.